

MDP 정식화 (Markov Decision Processes)

MDP의 다섯 요소와 마르코프 성질 · 누적 보상과 할인율 · 가치함수와 벨만 기대방정식

Machine Learning 2 · 3주차

한국과학영재학교 (KSA)

Chapter 3

이번 주차 개요

1950년대 벨만(Richard Bellman, 1957)은 “차원의 저주”라는 용어를 만들며 동적계획법을 고안. 핵심 통찰 — 복잡한 문제를 한 번에 풀지 말고, 작은 부분 문제로 쪼개 그 답을 재귀적으로 조합하라 — 가 지금 강화학습 이론 전체의 뼈대.

- Chapter 2의 밴딧은 상태가 정확히 하나뿐인, 전이도 없는 특수한 MDP.
- 이번 장은 밴딧에 상태와 전이를 더해, “지금 행동이 다음 상태를 바꾸고, 그 상태가 미래 보상을 결정한다”는 순환을 수학적으로 다룬다.

이 챕터를 마치면

- MDP의 다섯 요소 (S, A, P, R, γ) 를 한 줄로 쓰고, **마르코프 성질**이 무엇을 요구하는지 판단한다.
- 할인된 누적 보상(리턴) G_t 를 손으로, 코드로 계산하고, γ 이 드 언 굿(스려 자치 + 석계 벼스)을

세 차시

- ① **3.1 MDP의 다섯 요소와 마르코프 성질** – “게임의 규칙 전체”를 다섯 요소로 달고, 3칸 보드게임 MDP를 통째로 써본다.
- ② **3.2 누적 보상, 할인율, 그리고 실습** – 리턴 G_t 계산, 유효 시계, “ γ 가 목표를 바꾼다”를 확인한다.
- ③ **3.3 가치함수와 벨만 기대방정식** – V^π, Q^π 정의, 무한합을 한 스텝 재귀식으로 압축하고, 반복 대입을 예고한다.

이 “문법”을 확실히 잡아야, Ch. 4 정책평가 · Ch. 6 Q-러닝 · Ch. 9 DQN 손실함수가 사실

동일한 재귀식(벨만방정식)을 각각 다른 방식으로 계산, 함수치는 건위를 틈에 보아야 볼 수 있다.

3.1 MDP의 다섯 요소와 마르코프 성질

이 절에서 무엇을 만들 것인가

이 절은 두 가지를 만든다.

- 1 강화학습 문제를 **수학적으로 한 줄에** 쓰는 법 — MDP는 다섯 가지 요소 ($\mathcal{S}, \mathcal{A}, P, R, \gamma$) 하나로 “게임의 규칙 전체”를 담은 형식.
- 2 그 형식이 성립하려면 필요한 **마르코프 성질**의 정확한 의미와, 실제로는 언제 깨지는지(깨졌을 때 어떻게 다시 성립 시키는가).

이 두 가지는 앞으로 16개 챕터에서 매일 쓰게 될 **언어의 사전**이다.

- 벨만방정식(3.3), 정책평가(4.1), Q-러닝(6)의 모든 수식이 결국 $\mathcal{S}, \mathcal{A}, P, R$ 위에서 쓰인 것.
- 여기의 표기가 헛갈리면 뒤의 모든 것이 흐릿해진다.

MDP의 다섯 요소

MDP는 다섯 가지로 정의된다: $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$

요소를 한 줄로	의미
$\mathcal{S}$	가능한 상태 들의 집합
$\mathcal{A}$	가능한 행동 들의 집합
$P(s' s, a)$	s 에서 a 를 했을 때 s' 로 전이 될 확률
$R(s, a)$	s 에서 a 를 했을 때 받는 즉시 보상
$\gamma \in [0, 1)$	할인율 (discount factor)

Chapter 2의 밴딧과 비교

밴딧에는 $\mathcal{S}$ 도 P 도 **없었다**(팔을 당겨도 “다음 상태”가 없었다). MDP는 지금 한 행동이 다음 상태 $P(s'|s, a)$ 를 통해 **미래 전체**에 영향을 준다는 사실을 명시적으로 담는다.

“다섯 개면 충분한가?” — 상태가 화폐

MDP의 정의를 읽으면 “상태”가 모든 정보의 **화폐(currency)**로 등장한다.

- 보상은 (s, a) 에서, 다음 상태는 (s_t, a_t) 에서 결정된다.
- 즉 **상태만 알면 앞으로 일어날 것의 전체 확률법칙이 결정된다.**

이 문장이 바로 **“마르코프 성질”이라고 부르는 것의 정확하고 강한 형태**(다음 섹션에서 정식화).

다섯 요소가 “게임의 규칙 전체”를 담는다는 뜻

초기 상태 분포는 **알고리즘에 필요한 정보가 아니다** — 학습은 어떤 시작점에서든 똑같이 진행되며, 그 점은 연습문제 4에서 확인한다.

역사: 이름이 붙은 두 수학자, 반세기의 드라마

마르코프(Andrei Markov, 1856-1922)

- 확률론과 수론을 오가던 러시아 수학자.
- 1906년 논문: 당시 확률의 대명사 **베르누이 법칙** (독립 반복 시행의 상대빈도 $\rightarrow$ 확률)의 왕좌를 지키던 확률론에 **의존하는 시행**까지 포괄.
- “이번 결과가 직전 결과만 기억하고 이전은 잊는” 의존성(= 지금의 마르코프 체인)에서도 베르누이 법칙이 성립함을 보임.

의존성이 들어와도 법칙이 살려면 그 의존성은 **1단계만**이어야 한다는 발견이, 120년 뒤 강화학습에서 “상태 하나만 기억하면 된다”는 설계 원칙으로 직접 이어진다.

벨만(Richard Bellman, 1920-1984)

- 2차전지 중 미공군 폭격 편성 문제를 풀면서 발견: **최적 전략은 부분 문제의 최적 전략을 재귀적으로 조합할 수 있다**(최적성의 원리).
- 1957년 저서 Dynamic Programming에서 세운 프레임워크가 바로 MDP의 정식화.

흥미로운 우연: 마르코프 성질 논문(1906)과 MDP 정식화(1957) 사이는 50년, 그런데 두 사람의 문제는 **정확히 같은 문제의 양쪽 끝**이다.

같은 문제의 양쪽 끝

마르코프: “이 의존 구조가 어떤 **확률법칙**을 결정하는가”(기술, description)

벨만: “그 법칙 위에서 **최적 행동**을 어떻게 찾는가”(최적화, optimization)

이번 한기의 구조(이번 장 = 정식화 $\rightarrow$ Ch 4-6 = 최적화)가 이 드라마의 둘째막

한 편의 MDP를 통째로 쓰기: 3칸 보드게임

추상적으로 다섯 요소를 나열하는 것만으로는 체감되지 않는다. 아주 작은 MDP 하나를 **통째로** 써본다.

설정: 상태 0, 1, 2의 보드게임. 0에서 시작, 매 스텝 “오른쪽으로 한 칸” 또는 “자리참기”를 고름, 상태 2에 도착하면 **에피소드 종료**. 이동은 확률적: “오른쪽”을 골라도 0.3의 확률로 제자리에 미끄러진다.

- $\mathcal{S} = \{0, 1, 2\}$ — 보드의 세 칸. 2는 **터미널 상태**(terminal state, 게임 종료 상태).
- $\mathcal{A} = \{\text{right, wait}\}$ — 두 행동.
- $\gamma = 0.9$.

이 다섯 개(5-tuple)가 쓰이는 순간, 이 게임의 “규칙”은 **완전히** 정해졌다. 어떤 시작점에서든, 어떤 정책(상태마다 행동을 고르는 규칙)으로 놀든, 받을 보상 시퀀스의 **전체 확률분포**가 이 다섯 요소만으로 계산된다 — 게임에 “보이지 않는 상태”가 없어야 한다는 전제(= 마르코프 성질)를 붙이면 정확하다. 다음 장에서 벨만방정식으로 $V^\pi(s)$ 를 계산하면, 여기 쓴 P 와 R 이 그대로 수식에 대입된다.

3칸 보드게임: 전이확률 P 와 보상 R

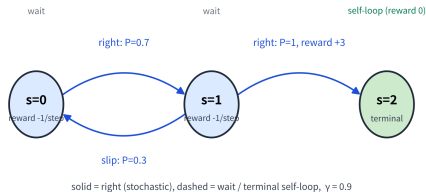
전이확률 P

상태	행동	s' (확률)
0	right	1 (0.7), 0 미끄러짐 (0.3)
0	wait	0 (1)
1	right	2 (1)
1	wait	1 (1)
2 (터미널)	(모든)	2 (1) self-loop

보상 R

- 터미널로 들어갈 전이(1에서 right): **+3**
- 그 외 모든 스텝: **-1** (한 스텝 움직이는 데 드는 “시간 벌금”)

3-state board-game MDP — the entire ruleset is captured by (S, A, P, R, γ)



3상태 보드게임 MDP — 상태 0에서 시작,
right(실선)/wait(점선) 전이, 미끄러짐 확률 0.3,
터미널 2에 진입 시 +3(그 외 매 스텝 -1).

각 요소 하나씩 (1/2): $\mathcal{S}$, $\mathcal{A}$

$\mathcal{S}$, 상태 공간

- 상태는 “에이전트의 관점에서의 **세계의 완전한 기술**”이어야 한다.
- CartPole의 상태 $[x, \dot{x}, \theta, \dot{\theta}]$ 가 4차원인 이유: 위치 두 개만 알고 속도 두 개를 모르면 “막대가 아직 기울어지고 있는가, 멈춰 있는가”를 분간할 수 없다.
- **상태에 빠진 정보는 에이전트에게 영구히 사라진 정보**다 — 보상이나 전이를 통해 돌아올 통로가 없기 때문.
- 이 “상태 설계”가 MDP를 세울 때 **가장 많은 시간이 드는 일**. 연속 상태 공간도 흔하다(CartPole처럼 $\mathcal{S} \subset \mathbb{R}^n$ 인 경우) — 표가 아니라 신경망으로 함수를 표현하게 된다(Ch. 9–11).

$\mathcal{A}$, 행동 공간

- 이산: CartPole의 left/right, FrozenLake의 4방향.
- 연속: Pendulum의 $[-2, 2] \subset \mathbb{R}$ (힘의 크기 그 자체를 고른다).
- 이산/연속의 차이는 곧 “어떤 함수를 표현할가”의 문제 — 이산이면 상태 $\times$ 행동 크기의 **표**, 연속이면 **연속 출력의 신경망**(Ch. 9 vs Ch. 10–11의 분기가 여기서 나온다).

각 요소 하나씩 (2/2): P, R, γ

P , 전이확률

- $P(s'|s, a)$ 는 “이 게임의 **물리 법칙**”이다.
- Gymnasium 환경에서는 이 함수가 `step()` 함수의 내부 구현으로 감춰져 있고, 우리는 **샘플**(실제 전이)만 관찰할 수 있다.
- **모델 기반**(Ch. 4-5 동적계획법): P 를 (추정해서) 명시적으로 써서 계획. **모델 없는**(Ch. 6-11): P 를 전혀 쓰지 않고 경험(실제 전이 샘플)에서만 가치를 배운다.
- 같은 P 라도 “정식적 표”로 줄 것이냐 “샘플 공급 원”으로 둘 것이냐가 알고리즘의 가족을 갈라놓는 **첫 번째 갈림길**.

R , 보상

- 수학적으로 더 정확한 표기는 **전이** $R(s, a, s')$ 에 붙는 것(같은 (s, a) 에서도 다음 상태가 다르면 보상이 다를 수 있음 — “도착했다/아직이다”는 s' 를 봐야 알 수).
- 보상의 **단위.스케일**은 알고리즘에 직접 영향을 준다: 10배 키우면 가치함수 전체가 10배가 되지만 **최적 정책은 변하지 않는다**(연습문제 5).
- 그러나 **상태/행동 사이에서 불균일**하게 들쭉한 스케일 은 조심해야 한다(Ch. 5에서 탐험 균형이 깨지는 사례).

마르코프 성질 (Markov property)

정의

마르코프 성질: 다음 상태는 오직 **현재** 상태와 행동에만 의존한다 — 어떻게 지금 상태에 도달했는지(과거 전체 이력)는 상관없다:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} \mid s_t, a_t)$$

- **예:** 체스판의 현재 배치만 알면, 거기까지 어떤 수순을 거쳐왔는지는 다음 수를 정하는 데 필요 없다.
- 이 성질 덕분에 “지금까지의 전체 이력”이 아니라 “현재 상태” **하나만 기억하면 충분해진다** — 뒤에서 배울 알고리즘들이 상태 하나만 보고 결정을 내릴 수 있는 이유가 여기서 나온다.

“상관없다”가 **확률적으로 정확히 무엇을 뜻하는가** — 조건부 확률의 정의와 사건의 독립성으로부터:

$$P(s_{t+1} = s' \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} = s' \mid s_t, a_t)$$

가 모든 과거 이력과 모든 다음 상태 s' 에 대해 성립할 때, s_t 가 (a_t 를 준 상태에서) **마르코프 성질**을 가진다고 한다.

왜 이 성질이 알고리즘의 모든 것을 결정하는가

마르코프 성질이 “좋은 가정”을 넘어서 필요 불가결인 이유. 성질이 없다면:

상태가치가 정의되지 않는다

- $V^\pi(s)$ 는 “상태 s 에서 시작”의 리턴 기댓값.
- “같은 s ”에 과거 이력이 다른 두 경로가 존재하고 그 미래 분포가 다르면, s 만으로 인자화한 하나의 함수 $V(s)$ 로 그 둘을 동시에 설명할 수 없다.
- 같은 자리에 “두 개의 다른 가치”가 공존하는데, 표 기반 알고리즘은 한 칸에 한 값만 쓸 수 있다.

학습이 수렴하지 않는다

- 같은 s 를 매번 다른 “진짜 상황”의 대리인으로 쓰면, 추정치는 서로 다른 답을 향해 계속 튀어 다닌다.
- Ch. 6-11의 모든 수렴 증명은 “같은 s 는 같은 미래 분포를 갖는다”는 이 동일성 위에서 있다.

반대로: 성질이 성립하면

상태 하나가 과거 전체를 압축하는 무손실 요약이 된다 — 통계학의 표현을 빌리면, 상태는 미래에 대한 과거 이력의 **충분 통계량(sufficient statistic)** 역할을 한다. 이 성질 덕분에 “기억의 길이”가 문제에서 빠져나가고, **현재 관측** 하나로 의사결정이 단한다.

마르코프 성질이 성립하지 않을 때: hidden state와 재설계

현실은 대부분 **부분관측**(partially observed)이다 — 에이전트가 보는 “관측”이 진짜 상태의 완전한 투영이 아니다.

대표 예: 자율주행 — 카메라 한 장만 관측으로 쓰면 “지금 얼마나 빠르게 움직이고 있었는가”라는 정보가 빠져, **같은 화면**에서 “속도가 높은 상황”과 “속도가 낮은 상황”이 구별되지 못한다 — 같은 s_{obs} 임에도 미래 분포가 **다르다** $\Rightarrow$ 성질이 깨진다.

상태 재설계의 두 가지 표준 전략

전략 1: 상태에 정보를 직접 넣는다 — 최근 k 프레임 $(o_t, o_{t-1}, \dots, o_{t-k+1})$ 을 하나의 “확장 상태”로 묶어 s_t 로 재정의. k 가 충분하면 **속도 정보가 (프레임 차이로) 복원**되어 성질이 다시 성립. 해석은 쉽지만 “ k 를 몇으로 할지”라는 **손으로 고르는 하이퍼파라미터**가 생긴다.

전략 2: 학습된 은닉 상태로 요약한다 — 신경망(예: RNN의 은닉 상태 h_t , Ch. 11)이 과거 관측 시퀀스를 h_t 로 압축하고 (o_t, h_t) 를 상태로 쓴다. “요약에 충분한 정보를 담았는지”는 수학적으로 검증할 수 없고 **경험적으로 판단**. 이것이 **부분관측 MDP(POMDP)** 라는, 이번 학기에서 직접 다루지는 않지만 **이름은 꼭 알아 둘 문제 영역**.

핵심

마르코프 성질은 “자연이 주는 성질”이 아니라, **우리가 상태를 어떻게 정의하는가의 설계 선택**이다. 성질이 “깨져” 보이면, 그건 우리의 상태 정의가 **부족**했다는 신호의 뿐. 강화학습 자체를 포기해야 할 이유가 아니다.

확인 문제: MDP인가, 마르코프인가? (1/2)

네 시나리오 각각에 대해 (a) MDP로 모델링할 수 있는가, (b) 가능하다면 $\mathcal{S}, \mathcal{A}, P, R$ 을 한 줄씩 쓰라:

- (a) 5층 건물의 엘리베이터 호출 제어

- ▶ 답: **MDP** — $\mathcal{S} = (\text{각 층의 상/하 호출 버튼 상태 } 2^{10} \text{ 조합}) \times (\text{엘리베이터 현재 층 } 5) = 5120 \text{ 개}$ 이하, $\mathcal{A} = \{up, down, open\}$, $P = \text{버튼 상태} + \text{층 이동}$, $R = -1$ (매 스텝 벌금) 또는 “누락 호출” 페널티.
- ▶ 마르코프 성질은 성립(현재 호출 상태가 미래의 모든 것을 결정).

- (b) “오늘 기분”을 몰라서, 어제 몇 시간 폰을 봤는 것만 관측으로 가지는 추천 문제

- ▶ 답: **마르코프 성질이 깨진** 문제 — “기분”이 **hidden state**.
- ▶ 어제 폰 사용 시간만으로는 같은 관측 아래 “피곤한 날”과 “재밌는 날”을 구별 못 해 **추천의 결과 분포가 달라진다**.
- ▶ 해결: 최근 며칠의 관측을 확장 상태로 묶거나 (전략 1), 사용자 모델의 은닉 상태로 요약(전략 2).

확인 문제: MDP인가, 마르코프인가? (2/2)

- (c) 슬롯머신(Chapter 2의 밴딧)

- ▶ 답: **MDP이지만 특수한 경우** — $\mathcal{S} = \{s_0\}$ (상태 1개), $P(s_0|s_0, a) = 1$ for all a .
- ▶ 밴딧은 “전이 없는 MDP”로, 이 절의 모든 정의가 **퇴화된 형태로** 적용(FAQ 참고).

- (d) 체스에서 다음 수를 고르는 문제(중반)

- ▶ 답: **MDP** — $\mathcal{S}$ = 합법적인 모든 판 배치(크기 $\sim 10^{40}$ 이상), $\mathcal{A}$ = 그 배치의 합법 수, P = 행동 후 판 배치(**결정적**이라 확률 1), R = 매 스텝 0, 체크메이트 시 ± 1 .
- ▶ 체스는 마르코프 성질이 **정확히** 성립하는 **희귀한 예** — “어떤 수순을 거쳐왔는지”가 다음 선택지에 영향을 주지 않으므로.

네 문항의 공통 기준

“**현재 상태(관측)만 알면, 앞으로 일어날 것의 전체 확률분포가 결정되는가?**” 아니라면 hidden state가 있다는 뜻이고, 상태 재설계(전략 1 또는 2)가 필요하다.

자주 하는 실수 (1/2): “관측”과 “상태”를 혼동

실습 코드에서 **가장 흔한 에러**: `obs`(환경이 돌려주는 관측)를 $\mathcal{S}$ 의 원소인 **상태**로 그대로 쓰고 마는 것. 두 개는 같은 환경에서 **같은 길이**를 가진 벡터일 수 있지만, **역할이 다르다**.

관측(observation)

- 에이전트가 볼 수 있는 것.
- 마르코프 성질을 보장하지 않는다 — Partially Observed MDP에서는 $o_t \neq s_t$ 다.
- 이번 학기의 Gymnasium 환경(CartPole, FrozenLake, Pendulum)은 **완전관측**이라 `obs`를 `s`로 써도 된다.
- **부분관측** 환경에서는 `obs`를 `s`로 쓰면 마르코프 성질이 깨져 **학습이 수렴하지 않거나, 수렴해도 잘못된 값**으로 수렴한다.
- “왜 내 에이전트가 수렴하지 않는가”를 debug할 때 **가장 먼저** 확인할 것: 현재 `obs`가 마르코프 성질을 만족하는가?

상태(state)

- 에이전트가 **알아야 하고, 알고리즘이 실제로 사용하는** 완전한 정보.
- 마르코프 성질이 **성립해야** 한다.

자주 하는 실수 (2/2): P 를 “한 번의 샘플”로 혼동

두 번째 흔한 실수: P 를 “한 번의 샘플”로 혼동.

- $P(s'|s, a)$ 는 확률 **분포**이지 한 번의 결과가 아니다.
- (0에서 right)을 100번 반복하면 **70번 1로, 30번 0으로** 갈 것이지만, 100번 중 한 번의 샘플이 “1로 갔다”는 사실은 $P = 0.7$ 을 **증명하지 않는다** — 추정치일 뿐이다.
- **모델 기반** 알고리즘(Ch. 4-5)은 이 추정치의 정확도에 **민감**하므로, 샘플 수와 추정 오차의 관계를 Chapter 5에서 자세히 다룬다.

요약

“관측 $\neq$ 상태”(역할의 차이), “ $P \neq$ 한 번의 샘플”(분포 vs 단일 결과). 두 실수는 모두 **확률적 MDP의 표기와 부분관측**을 혼동하는 것에서 온다.

실습: Gymnasium에서 $\mathcal{S}$, $\mathcal{A}$ 확인

이론의 다섯 요소를 실제 환경의 API와 대응시킨다. FrozenLake-v1(얼음 위를 미끄러지지 않게 건너는 문제)과 CartPole-v1 을 만들고, 각각의 $\mathcal{S}$ 와 $\mathcal{A}$ 를 출력:

```
import gymnasium as gym

# --- FrozenLake: 이산상태행동 · 공간 ---
fl = gym.make("FrozenLake-v1", is_slippery=True)
obs, info = fl.reset(seed=0)
print("FrozenLake S:", fl.observation_space)    # Discrete(16)
print("FrozenLake A:", fl.action_space)         # Discrete(4)
print("초기 상태:", obs, "(4x4 격자칸 16 중하나)")
fl.close()

print()

# --- CartPole: 연속상태 , 이산행동 ---
cp = gym.make("CartPole-v1")
obs, info = cp.reset(seed=0)
print("CartPole S:", cp.observation_space)      # Box(4,)
print("CartPole A:", cp.action_space)           # Discrete(2)
print("초기 상태:", obs, "[x, x_dot, theta, theta_dot]")
cp.close()
```

실습: 마르코프 성질을 샘플링으로 직접 검증

마르코프 성질이 “성립한다”는 말이 **검증 가능한 수학적 주장**인지 직접 확인. 간단한 결정적 환경(2-경로 환경: 상태 0에서 행동 0이면 1로, 행동 1이면 2로 전이)을 만들고, **같은 상태 1에 다른 경로**($0 \rightarrow 1$ vs $0 \rightarrow 2 \rightarrow 1$)로 도달한 뒤, 다음 상태 분포가 같은지 샘플링으로 확인:

```
import numpy as np

class TwoPathsEnv:
    """0 -> {1, 2} -> 1 -> 1(self-loop) 결정적 MDP."""
    def __init__(self, seed=0):
        self.rng = np.random.default_rng(seed)
        self.s = 0
    def reset(self, seed=None):
        if seed is not None:
            self.rng = np.random.default_rng(seed)
        self.s = 0
        return self.s
    def step(self, a):
        if self.s == 0:
            self.s = 1 if a == 0 else 2
        elif self.s == 1:
            self.s = 1 # self-loop, 경로무관
        else:
            self.s = 1 if a == 0 else 2
```

마르코프 검증: 핵심 패턴과 오차 분석

핵심 패턴: “같은 상태, 다른 경로, 다음 분포 비교”

이 패턴이 마르코프 성질 검증의 표준 절차다. 실습 노트북에서 이산/연속 환경 모두에서 더 자세히 한다.

- 결정적 환경에서는 차가 정확히 0.
- 확률적 환경에서는 샘플링 오차 ($\sim 1/\sqrt{N}$)가 있다.
- $N = 10000$ 이면 오차는 약 **0.003** 정도로, 0.01 임계값 안에 들어온다.

“마르코프 성질 성립”은 “같은 상태 s_t 에 서로 다른 경로로 도달했을 때, 다음 상태 분포가 같다”는 검증 가능한 수학적 주장이다. 결정적 환경에서는 차이 = 0, 확률적 환경에서는 $\sim 1/\sqrt{N}$ 샘플링 오차 안에서 0에 수렴한다.

실습: FrozenLake에서 P 를 실제로 추정하기

Gymnasium 환경의 P 는 내부 구현으로 감춰져 있지만, **샘플링으로 추정**할 수 있다:

$$P(s'|s, a) \approx \frac{N(s, a, s')}{N(s, a)}$$

((s, a) 에서 s' 로 전이한 횟수 / (s, a) 총 시도 횟수)로, 무작위 행동을 많이 하면 경험적 전이확률에 수렴.

```
import gymnasium as gym
import numpy as np

env = gym.make("FrozenLake-v1", is_slippery=True)
env.action_space.seed(0)
n_states = env.observation_space.n # 16
n_actions = env.action_space.n # 4
# count[s][a][s'] = (s,a)->s' 전이횟수
count = np.zeros((n_states, n_actions, n_states), dtype=int)
```

```
for ep in range(2000):
    s, _ = env.reset(seed=ep)
    done = False
    while not done:
```

```
        a = env.action_space.sample()
        s2, r, term, trunc = env.step(a)
```

P 추정 결과: 모델 기반 vs 모델 없는

2000에피소드의 무작위 탐색으로 추정된 P 는, 실제 FrozenLake 구현(is_slippery=True일 때 intended 방향 2/3, 좌/우 1/6씩)에 수렴.

핵심: “ P 의 사용 방식”이 알고리즘의 가족을 나눈다

이 P_{est} 를 Chapter 4의 동적계획법에 그대로 대입하면 **모델 기반** 알고리즘을 돌릴 수 있고, P 를 무시하고 **경험만** 쓰면 **모델 없는** 알고리즘(Ch. 6-)을 돌릴 수 있다.

같은 환경, 같은 샘플, 다른 “ P 의 사용 방식”이 알고리즘의 가족을 나눈다.

모델 기반(Ch. 4-5): 추정된 P 를 명시적으로 써서 계획. 모델 없는(Ch. 6-11): P 를 전혀 쓰지 않고 경험(실제 전이 샘플)에서만 가치를 배운다. 이 “첫 갈림길”이 이 학기의 알고리즘 지도를 결정한다.

이 절을 마무리하며: 강화학습의 “문법”

이 절에서 만든 것을 한 줄로 요약:

$$\text{강화학습 문제} = (\mathcal{S}, \mathcal{A}, P, R, \gamma) + \text{마르코프 성질}$$

이 형식이 성립하면:

- ① 가치를 $V^\pi(s)$ 와 같은 “**상태에만 인자화된 함수**”로 정의할 수 있고(3.3),
- ② **벨만방정식**이라는 한 스텝 재귀식으로 가치를 계산할 수 있고(3.3),
- ③ 정책평가 · Q-learning · DQN · PPO 까지 — **이 형식 위에서 쓰인 모든 알고리즘을** 도출할 수 있다.

마르코프 성질이 “깨진” 문제(POMDP)

이 “문법”의 전제가 깨진 것이므로, “상태를 재설계해서 성질을 다시 성립시키는 것”이 **항상 첫 번째 해결책**.

FAQ (1/2)

● Q. 마르코프 성질이 실제 문제에서 항상 성립하나요?

- ▶ 엄밀하게는 거의 항상 근사.
- ▶ 예: 자율주행에서 “카메라 한 장”만 상태로 쓰면 **속도·가속도 정보가 빠져** 마르코프 성질이 깨진다 (다음 상태를 예측하려면 “지금 얼마나 빠르게 움직이고 있었는가”가 필요한데, 정지 이미지 한 장에는 그 정보가 없다).
- ▶ 실전: **최근 몇 프레임을 함께 묶어 “상태”로 재정의**하거나, **신경망이 과거 정보를 요약한 은닉 상태**를 상태의 일부로 쓰는 방식 (Ch. 11 RNN 은닉 상태와 같은 아이디어)으로 다시 성립.

● Q. 밴딧(Ch. 2)도 MDP의 특수한 경우인가요?

- ▶ 그렇다 — $\mathcal{S} = \{s_0\}$ (상태 하나)에서 어떤 행동을 해도 **항상 같은 상태로 “전이”**하는 특수한 MDP.
- ▶ Ch. 2.3에서 밴딧과 MDP를 “별도”로 소개했지만, 이렇게 보면 이번 강의 **모든 정의**(가치함수, 벨만방정식)가 밴딧에도 **되화된 형태로 그대로 적용**.

FAQ (2/2)

● Q. 보상을 10배 키우면 최적 정책이 바뀌나요?

- ▶ 바뀌지 않는다 — 모든 $Q^\pi(s, a)$ 가 10배가 되지만, $\arg \max_a Q^\pi(s, a)$ 는 변하지 않음.
- ▶ 그러나: 보상의 상수 offset(모든 보상에 같은 상수 c 를 더하기)도 최적 정책을 바꾸지 않음(모든 Q 에 $c/(1 - \gamma)$ 가 균일하게 더해져 argmax 보존).
- ▶ 반면 상태/행동별로 불균일하게 보상을 조정하면(예: 한 상태에만 벌금 추가) 정책이 바뀔 수 있다.
- ▶ 보상의 “스케일”은 안전, “형태”는 위험하다.

● Q. γ 가 MDP의 “다섯 요소” 중 하나인데, 왜 3.2에서 자세히 다루나요?

- ▶ γ 의 수학적 역할(무한급수 수렴 보장, 가치함수 유계 보장)은 이번 절의 정의에서 이미 고정되어 있다 — $\gamma \in [0, 1)$ 으로 써둔 것이 그 전부.
- ▶ γ 의 실무적 의미(할인율, “지금 vs 미래” 교환 비율, 0.9~0.99로 고르는 기준)은 3.2에서 누적 보상 G_t 의 정의를 세울 때 자연스럽게 나온다.

● Q. 터미널 상태(terminal state)는 MDP의 다섯 요소에서 어디에 들어가는가?

- ▶ 터미널 상태는 $\mathcal{S}$ 의 원소지만, 전이 행동이 제한된 특수한 상태 — 보통 **self-loop**(자기 자신으로 확률 1 전이)로 표현하고, **보상 0**을 준다.
- ▶ “종료”의 의미를 MDP 형식 안에 넣는 **표준 방식**이 바로 이것.
- ▶ Ch. 4 정책평가 코드에서 볼 수 있듯, 터미널 상태를 self-loop으로 표현하면 벨만방정식이 터미널에서도 **같은 형태**($V(s_{\text{term}}) = 0$)를 유지해 **코드 분기가 필요 없다**.

● Q. Gymnasium의 terminated 와 truncated 는 MDP의 어떤 것에 해당하는가?

- ▶ terminated = **진짜 종료**(터미널 상태 도달, 게임 끝) → MDP의 **터미널 상태**와 정확히 대응.
- ▶ truncated = **시간 한도 도달**(게임은 계속됐을 것이지만 학습 편의상 끊음) → MDP 형식에는 **직접 대응하지 않는 학습 장치**.
- ▶ 가치함수 업데이트 시 terminated 는 “**미래 가치 0**”로, truncated 는 “**다음 상태의 가치를 그대로**”로 처리해야 한다는 차이(1.3절에서 봤듯)가 여기에서 온다.

3.2 누적 보상, 할인율, 그리고 실습

이 절의 흐름

3.1절에서 MDP의 다섯 요소 ($\mathcal{S}, \mathcal{A}, P, R, \gamma$) 를 “나열”하기만 했다. 이 절은 그중 R (보상)과 γ (할인율)에 집중해서, 강화학습이 실제로 **최대화 하는 대상 — 리턴 — 을 손으로 코드로 계산할 수 있게 만든다.**

수업의 흐름:

- ① **리턴**(할인된 누적 보상)의 정의와 유한/무한 경우의 계산법.
- ② “할인율 0.95”가 실제로는 “몇 스텝짜리 시야”를 뜻하는지 — **유효 시계(effective horizon)**로 바꾸는 법.
- ③ γ 를 바꾸면 “같은 환경에서 다른 행동을 최적으로” 만드는 예 — 할인율이 단순 정규화 장치가 아니라 목표를 바꾸는 매개변수임을 확인.
- ④ Gymnasium에서 CartPole로 할인된 리턴을 실제로 누적.

리턴(return)의 정의

강화학습의 목표는 한 스텝의 보상이 아니라, 앞으로 받을 **보상들의 합** — **리턴(return)** G_t — 을 최대화하는 것이다:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

왜 할인율 γ 가 필요한가

- ① **수학적으로:** $\gamma < 1$ 이면 보상이 유계(bounded)인 한 이 **무한급수가 항상 수렴**(기하급수).
 $\gamma = 1$ 이면 끝나지 않는 과업에서 리턴이 **무한대로 발산**할 수 있다.
 - ② **직관적으로:** “지금 당장의 보상 1”이 “10스텝 뒤의 보상 1”보다 더 가치 있다고 보는 경우가 많다(사람의 시간 선호, 금융의 이자율과 같은 방향).
- γ 가 0에 가까우면 **근시안적**(당장의 보상만 중시), 1에 가까우면 **장기적**(먼 미래까지 충분히 고려).

닫힌 형태로 잘 풀리는 무한합

이 무한합은 실제로 **닫힌 형태로 잘 풀린다**. $\gamma < 1$ 이면 기하급수의 합 공식을 그대로 쓸 수 있다.

보상이 매 스텝 같은 값 R 으로 고정된 경우:

$$G_t = R + \gamma R + \gamma^2 R + \cdots = \frac{R}{1 - \gamma}$$

- $\gamma = 0.9, R = 1$ 이면 $G_t = \frac{1}{0.1} = 10$ — “매 스텝 1을 무한히 받는 환경”의 리턴이 10이라는 뜻.
- γ 가 1에 다가갈수록 이 값은 $\frac{1}{1-\gamma}$ 처럼 **무한대로 올라간다** — “발산”의 구체적 모양.

보상이 매 스텝마다 다른 일반적 경우: 한 에피소드가 유한한 스텝 T 뒤에 끝나면(터미널 도달 뒤 보상은 모두 0) **무한합은 유한합으로 잘린다:**

$$G_0 = \sum_{k=0}^T \gamma^k R_k = R_0 + \gamma R_1 + \gamma^2 R_2 + \cdots + \gamma^T R_T$$

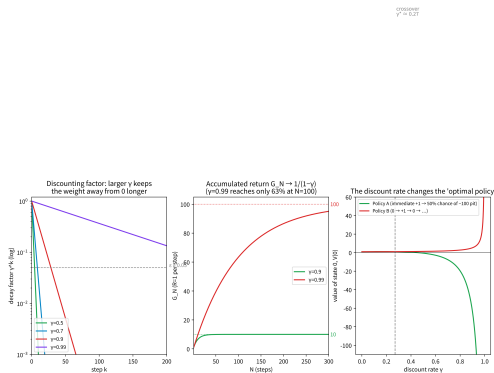
“무한합”이라는 **모양만** 무서울 뿐, 실제로는 **마지막 보상 R_T 의 계수 γ^T 가 아주 작으므로** 유효한 항은 앞에 몇 개뿐이다 — “몇 개쯤이 유효한지”를 숫자로 잡는 것이 바로 다음 절의 **유효 시계다**.

매 스텝 보상 $R = 1$ 일 때: 유한합이 닫힌 형태로 접근

매 스텝 보상 $R = 1$ 일 때, 유한합 $G_N = \sum_{k=0}^{N-1} \gamma^k$ 이 닫힌 형태 $\frac{1}{1-\gamma}$ 로 어떻게 다가가는지:

N	$G_N (\gamma = 0.9)$	$G_N (\gamma = 0.99)$
5	4.10	4.90
10	6.51	9.56
20	8.78	18.21
50	9.95	39.50
100	10.00	63.40
∞	10	100

- $\gamma = 0.9$ 는 20스텝이면 이미 닫힌 형태의 **87.8%**, 100스텝이면 99.997%.
- $\gamma = 0.99$ 는 100스텝이 돼도 **63.4%**에 불과.



(가운데) 매 스텝 보상 1의 누적 리턴이 $1/(1-\gamma)$ 로 수렴.

결론

γ 가 클수록 합산해야 하는 항의 수가 늘어 “더 장기적”이라는 말의 구체적 수학적 의미.